

Chapter 3: Methodology and Implementation

3.1 Introduction

This chapter outlines the approach taken to design, develop, and assess a deepfake detection system for audio-visual content using the AV-Deepfake1M++ dataset (Cai et al., 2025). The aim is to meet the research goals systematically while respecting hardware, storage, and computational constraints.

The work targets content-driven deepfakes where a real identity is preserved but the spoken content is altered via audio synthesis and lip synchronisation, alongside visual manipulations. Such changes are subtle and localised, making unimodal detection unreliable (Cai et al., 2024; Korshunov and Marcel, 2018). A multimodal strategy is therefore adopted that exploits inconsistencies between the audio stream and the visual stream.

This chapter also documents the deviations from the initially proposed architecture and the practical reasons that necessitated each change. The initial proposal outlined a system built around Wav2Vec 2.0 for audio feature extraction, MobileNetV3 for visual feature extraction, and a DiMoDif fusion module. As implementation progressed, each of these components was revised in response to engineering constraints, training instability, and the specific characteristics of the dataset. The rationale for each departure is discussed explicitly alongside the adopted solution.

3.2 Research and Development Approach

This study uses a quantitative research approach based on secondary analysis of the AV-Deepfake1M++ dataset (Cai et al., 2025). Using an existing dataset avoids the cost and practical difficulties of collecting a large-scale audio-visual corpus, which would not be feasible within the scope of this project.

To manage development and experimentation, an incremental workflow was adopted. Each component — data loading, preprocessing, feature extraction, model integration, and evaluation — was implemented and verified independently using a small video subset before integration into the full pipeline. This approach reduced debugging complexity and avoided unnecessary computation when working with high-dimensional audio and video data.

All hyperparameters and derived constants were centralised in a single configuration file (`config.py`) from the outset, ensuring that no values were hardcoded across model or training files. This made systematic experimentation and iterative improvement tractable, as any change to the extraction parameters, model dimensions, or training schedule could be made in one place and propagated automatically throughout the pipeline.

3.3 System Architecture and Implementation

3.3.1 Overview of the Proposed System

The primary objective of the project is to maximise classification accuracy at the clip level across all four manipulation types: `real`, `audio_modified`, `visual_modified`, and `both_modified`. Although the AV-Deepfake1M++ dataset provides temporal localisation annotations, frame-level modelling was not adopted in this implementation due to the high computational cost and storage demands it would introduce. The system therefore operates at the video-clip level.

The implemented system is a Cross-Modal Transformer Fusion network that combines two pretrained modality-specific encoders with a learned cross-modal attention mechanism, and produces three simultaneous binary predictions per video clip: whether the audio is authentic, whether the video is authentic, and an overall joint verdict. This multi-head design, rather than a single output, allows the model to independently specialise each head on the evidence available from each modality, while the joint head captures cross-modal interaction.

3.3.2 Dataset Selection and Subset Strategy

The AV-Deepfake1M++ dataset is approximately 1.4 TB in size, which makes full local storage and processing infeasible within the available development environment. The validation split, comprising 77,326 video clips, was used exclusively throughout this work. Of these, 68,851 videos (89%) were confirmed present on disk following extraction; the remaining 8,475 were absent due to corruption or incomplete extraction and were excluded from all experiments. The four manipulation categories in the validation split are balanced, each containing between 16,848 and 18,037 videos, as shown in Table 3.1.

Table 3.1: Validation split composition

Category	Description	Count
<code>real</code>	Unmodified audio and video	18,037
<code>audio_modified</code>	Voice replaced or cloned; video untouched	16,848
<code>visual_modified</code>	Face swapped or reenacted; audio untouched	17,020
<code>both_modified</code>	Both audio and video manipulated	16,946

Each entry in the accompanying metadata file (`val_metadata.json`) records the file path, manipulation type, frame counts, and the temporal coordinates of any manipulated segments (`fake_segments`) as `[start_sec, end_sec]` pairs.

3.3.3 Audio Feature Extraction Module

Initial Proposal

The initial proposal specified Wav2Vec 2.0 (Baevski et al., 2020) as the audio feature extraction backbone. Wav2Vec 2.0 is a self-supervised speech model that learns contextual speech representations directly from raw waveforms, capturing prosodic and phonemic information that handcrafted features such as MFCCs would miss. Its contextual embeddings were expected to be particularly sensitive to the subtle artefacts introduced by modern voice cloning and text-to-speech synthesis systems.

Change Applied and Rationale

During implementation, Wav2Vec 2.0 was replaced with a ResNet18 (He et al., 2016) backbone pretrained on ImageNet, applied to mel-spectrogram representations of the audio signal.

This change was driven by three practical constraints. First, Wav2Vec 2.0 produces variable-length sequence outputs whose length depends on the duration of the input audio. Integrating this with a fixed-dimension video feature vector inside a Transformer fusion module required either temporal pooling — which discards the temporal structure that motivates using Wav2Vec in the first place — or padding and masking strategies that added architectural complexity without clear benefit at the clip level. Second, the Wav2Vec 2.0 large model imposes a substantial VRAM footprint. When combined with ResNet3D-18 for video encoding and a Transformer fusion module, the total memory requirement exceeded the GPU capacity of the available training hardware during early batch-size experiments. Third, torchaudio’s robust FFmpeg-native audio loading pipeline, combined with the MelSpectrogram and AmplitudeToDB transforms, provided a stable and crash-free audio extraction pathway that was compatible with the corrupted and non-standard MP4 files common in the dataset. Earlier attempts using librosa-based loading produced frequent `PySoundFile failed` warnings and occasional crashes on corrupted video files, which disrupted the parallel extraction pipeline.

The mel-spectrogram representation converts raw audio into a 2D time-frequency image of shape $(1, 128, 63)$, which can be processed by any image CNN with a simple modification to the first convolutional layer to accept a single channel rather than three. Voice cloning artefacts, unnatural harmonic structures, and audio splice boundaries all manifest as visible patterns in the mel-spectrogram that a CNN trained on general image representations can detect. This approach is consistent with established practice in audio classification research, where ImageNet-pretrained CNNs applied to spectrograms have repeatedly matched or outperformed bespoke audio architectures on downstream tasks.

Audio was sampled at 16,000 Hz. A 1024-point FFT with hop length 512 and 128 mel frequency bins was applied, yielding the $(1, 128, 63)$ spectrogram. Amplitude was converted to decibels with an 80 dB dynamic range, and each spectrogram was per-sample normalised to zero mean and unit variance. All extraction parameters were read from `config.py` rather than hardcoded,

ensuring that changing the FFT window or mel bin count propagated automatically to the derived `target_t` dimension.

3.3.4 Visual Feature Extraction Module

Initial Proposal

The initial proposal specified MobileNetV3 (Howard et al., 2019) as the visual backbone, focused on the mouth region of interest (ROI). Facial landmark detection was to be used to crop the lip region prior to encoding. MobileNetV3 was selected for its lightweight architecture, which offers a practical balance between efficiency and representational capacity when frame-level spatial features are required.

Change Applied and Rationale

During implementation, MobileNetV3 applied to lip-region crops was replaced with ResNet3D-18 (Tran et al., 2018) pretrained on Kinetics-400 (Kay et al., 2017), applied to full 224×224 frames across the full temporal window of 50 frames.

There were two primary motivations for this change. First, the initial proposal’s mouth-ROI approach assumed that deepfake artefacts are localised to the lip region. However, the `visual_modified` and `both_modified` categories in AV-Deepfake1M++ encompass a range of face-swap and reenactment techniques where artefacts are distributed across the entire face — including skin texture boundaries, hairline artifacts, and blending inconsistencies at the face perimeter — none of which are captured by a lip-region crop. Restricting the visual field to the mouth region would systematically discard evidence that the visual encoder needs to detect these manipulation types.

Second, applying MobileNetV3 frame-by-frame produces independent spatial features for each frame with no temporal context. Deepfake videos frequently exhibit temporal inconsistencies — unnatural head motion, inconsistent blinking, or jittery texture between consecutive frames — which are not visible in any single frame but emerge clearly when multiple frames are considered jointly. ResNet3D-18’s 3D convolutional filters jointly convolve the spatial and temporal dimensions, capturing exactly these cross-frame patterns. As Dolhansky et al. (2020) and Rossler et al. (2019) both note, temporal modelling is essential for robust video-level detection, particularly as generative models improve and per-frame artefacts become less pronounced.

Fifty frames were sampled from each two-second window at 25 frames per second. Frames were resized to 224×224 pixels and normalised using ImageNet statistics. Data augmentation during training — random horizontal flipping and brightness and contrast jitter — was applied before ImageNet normalisation to keep pixel values in the valid $[0, 1]$ range prior to the normalisation step. The resulting video tensor has shape $(50, 3, 224, 224)$.

3.3.5 Cross-Modal Fusion and Classification

Initial Proposal

The initial proposal specified the DiMoDif architecture (Cai et al., 2025), which is designed specifically for the AV-Deepfake1M++ dataset. DiMoDif models fine-grained phoneme-to-viseme alignment between audio and visual streams, directly targeting content-driven deepfakes where speech content is synthesised and lip motion is generated to match. The temporal boundary detection components of DiMoDif were to be excluded in favour of video-level classification.

Change Applied and Rationale

DiMoDif was replaced with a custom two-layer Transformer Encoder fusion module with a learnable [CLS] token, operating on projected audio and video feature vectors.

The primary reason for this departure was reproducibility and implementation complexity. DiMoDif requires precise temporal alignment between audio phoneme sequences and per-frame visual features, which presupposes that both streams can be reliably aligned at the sub-frame level. Achieving this alignment robustly across the full diversity of video codecs, frame rates, and audio sampling rates present in the dataset would have required significant additional preprocessing infrastructure. Given that the temporal boundary detection components of DiMoDif were explicitly excluded from the proposal, retaining only the fusion mechanism while discarding its motivating temporal alignment would have reduced DiMoDif to a cross-modal attention module — which is precisely what the implemented Transformer fusion provides, with fewer implementation dependencies.

The Transformer Encoder fusion module receives the 256-dimensional feature vectors produced by both encoders, projects each to 512 dimensions, and forms a three-token input sequence [CLS, video, audio] augmented with learned positional embeddings. Two layers of multi-head self-attention with eight heads, GELU activation, and pre-norm layer normalisation allow the video and audio tokens to attend to each other. This attention mechanism captures cross-modal inconsistencies — for example, audio spectrogram patterns that do not correspond to the observed facial motion — in a way that simple feature concatenation and MLP fusion cannot, as noted by Yi et al. (2023) in their survey of multimodal detection approaches. The [CLS] token output aggregates information from both modalities and feeds three independent sigmoid classification heads: one for the audio stream, one for the video stream, and one for the joint verdict.

On CPU-only hardware, the Transformer module is automatically replaced with a lightweight MLP fusion module to reduce inference latency, making the system deployable without GPU hardware.

3.4 Model Training and Evaluation Strategy

3.4.1 Loss Function

The initial proposal used standard Binary Cross-Entropy (BCE) loss. In the implemented system, this was replaced with Focal Loss (Lin et al., 2017):

$$\mathcal{L}_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

where $\gamma = 2.0$ is the focusing parameter and $\alpha = 0.25$ is a class balance weight. The motivation for this change arose from observations during early training runs in which the model rapidly converged to a state of predicting the majority class for the majority of examples. With four balanced manipulation types, easy examples — videos with obvious, high-contrast artefacts — dominated the gradient signal and prevented the model from learning to detect subtle manipulations. Focal Loss downweights the contribution of easy, well-classified examples through the $(1 - p_t)^\gamma$ term, concentrating training capacity on hard, ambiguous cases. When $\gamma = 0$, Focal Loss reduces to standard BCE, so the change is strictly a generalisation of the original proposal.

The total loss combines all three classification heads:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{audio}} + \mathcal{L}_{\text{video}} + w_j \cdot \mathcal{L}_{\text{joint}}$$

The joint head is weighted by $w_j = 2.0$ to reflect its role as the primary detection target. Both γ and w_j are configurable through `config.py`.

3.4.2 Two-Phase Training

Training proceeded in two phases to protect the pretrained encoder features during early optimisation. In Phase 1, covering the first 25% of training epochs, the video and audio encoder parameters were frozen and only the fusion module and classification heads were updated. This prevented the randomly-initialised fusion module from generating large, destructive gradients that could corrupt the Kinetics-400 and ImageNet pretrained weights before any meaningful fusion representations had been established.

In Phase 2, covering the remaining 75% of epochs, all parameters were unfrozen. Encoder parameters were trained at a learning rate of 1×10^{-5} , ten times lower than the fusion module’s 1×10^{-4} , to allow gradual domain adaptation without catastrophic forgetting of pretrained features. Both the phase boundary and the patience threshold for early stopping were expressed as fractions of the total epoch count (`freeze_epochs = max(1, round(epochs × 0.25))`, `patience = max(5, round(epochs × 0.30))`), making the training schedule scale automatically with any change to the total epoch budget.

3.4.3 Optimiser and Scheduler

AdamW was used with weight decay 1×10^{-4} . Gradient norms were clipped to a maximum of 1.0 per step to prevent instability arising from the 3D convolutional operations on 50-frame inputs, which can produce large gradient magnitudes. ReduceLROnPlateau halved all learning rates when validation joint AUC did not improve for five consecutive epochs, allowing the model to settle into finer minima as training progressed. Early stopping halted training when no improvement was observed for 30% of the total epoch budget.

Multi-GPU training was managed via PyTorch `DataParallel`, with the effective batch size scaling linearly with the number of available GPUs. This was necessary because the GPU instances available through Vast.ai varied in configuration between training runs.

3.4.4 Checkpoint Management and Resumability

The model state, optimiser state, learning rate scheduler state, and random number generator states (Python, NumPy, and PyTorch) were all saved to disk at the end of every epoch. The checkpoint achieving the highest validation joint AUC was saved separately as `best_model.pth`. A separate `training_checkpoint.pth` stored the latest epoch state for resuming interrupted runs.

This design was motivated by the use of cloud-based GPU instances, where sessions are time-limited and instances can be terminated unexpectedly. The resumable pipeline ensured that no completed work was lost, and that training could continue from the exact state — including random seeds — at which it was interrupted. The `best_model.pth` file is the checkpoint used for all evaluation and inference; `training_checkpoint.pth` is used only to resume training.

3.5 Challenges and Limitations

The primary limitation of this study arises from computational and storage constraints. Using only the validation split of AV-Deepfake1M++ rather than the full training set limits the diversity of manipulation techniques and speakers to which the model is exposed during training. The training and validation subsets were constructed from this single split using a speaker-based partition, which ensures that the evaluation is honest but means the effective training set is smaller than would be ideal.

A second limitation is the exclusion of temporal localisation. Although frame-level manipulation detection could provide more granular insight into model behaviour and enable localisation of the manipulated region within a clip, this was deferred due to the computational requirements of frame-level modelling and the storage demands of per-frame feature representations.

Third, the audio pipeline processes a fixed two-second window rather than the full clip. For videos where the manipulated segment is long or distributed across the clip, the selected window

may not always capture the most informative region, particularly for real videos where the central two seconds are used by default.

Finally, since the dataset was constructed by third parties, the study relies on the accuracy and completeness of the existing annotations. Any labelling bias or inconsistency in the `fake_segments` annotations may influence model performance in ways that are not easily detectable.

3.6 Ethical Considerations

The dataset used in this study is publicly available and was collected under established data use agreements. As such, no direct ethical concerns arise from data collection or use. However, deepfake detection research carries broader ethical implications. Although the goal of this work is defensive in nature — developing tools to identify manipulated media — the same techniques and architectural insights could theoretically inform improvements to generative systems (Chesney and Citron, 2019; Westerlund, 2019). This study positions its contribution strictly within the context of detection and harm mitigation, without making deployment or real-world enforcement claims.

3.7 Implementation Details

All experiments were implemented in Python 3.12 using PyTorch 2.0. Key libraries included TorchVision for the ResNet3D-18 and ResNet18 architectures, TorchAudio for audio loading and mel-spectrogram computation, OpenCV for video frame extraction, and scikit-learn for dataset partitioning and metric computation. Training was conducted on NVIDIA GPU servers accessed via Vast.ai. Experiment tracking, metric logging, and training visualisation were managed using Weights & Biases.

The full pipeline — from data download through evaluation — is implemented as a modular codebase of nine Python files, each with a clearly defined responsibility, as shown in Table 3.2.

Table 3.2: Codebase module responsibilities

Module	Responsibility
<code>config.py</code>	All hyperparameters, paths, and derived constants
<code>data_utils.py</code>	Metadata loading, speaker split, parallel feature extraction, dataset class
<code>audio.py</code>	Audio encoder (ResNet18 on mel-spectrogram)

Module	Responsibility
<code>video.py</code>	Video encoder (ResNet3D-18 on frame sequences)
<code>cross_modal.py</code>	Fusion module (Transformer and MLP variants)
<code>train_utils.py</code>	Training loop, validation, loss, optimiser, W&B logging
<code>checkpoint_utils.py</code>	Checkpoint save and load
<code>main.py</code>	Pipeline entry point
<code>inference.py</code>	Standalone single-video and batch inference

Parallel feature extraction used Python’s `multiprocessing.Pool` with fork context and up to 28 CPU workers. Each video’s features were saved as an individual `.pt` file to avoid loading the full dataset into RAM, which would have required several terabytes of memory. A manifest JSON file indexed all successfully extracted samples and was checkpointed every 500 videos to support crash recovery. Corrupted or unreadable videos were tracked in a separate failed-samples list and skipped on subsequent runs.

3.8 Conclusion

This chapter has described the complete methodology and implementation of the audio-visual deepfake detection pipeline, from data acquisition through model training and evaluation. The implemented system diverges from the initial proposal in three principal ways: the audio encoder was changed from Wav2Vec 2.0 to a ResNet18 applied to mel-spectrograms, due to memory constraints and compatibility issues with the parallel extraction pipeline; the visual encoder was changed from a lip-region MobileNetV3 to a full-frame ResNet3D-18, to capture temporal artefacts and whole-face manipulation patterns; and the DiMoDif fusion module was replaced with a custom Transformer Encoder fusion, to avoid the temporal alignment requirements of DiMoDif while preserving the cross-modal attention capability that motivates its design. Standard BCE loss was additionally replaced with Focal Loss to address gradient domination by easy examples during training. Each of these changes is grounded in specific implementation constraints or empirical observations encountered during development. The results of applying this pipeline are presented in Chapter 4.